

Sistema de Chat Distribuído

Relatório do Trabalho Final — Sistemas Distribuídos

Aluno: Matheus Costa de Menezes **Disciplina:** Sistemas Distribuídos — CEFET-MG — 2026/1

Professora: Michelle Hanne **Código-fonte:** <https://github.com/MatheusMnz/ChatDistribuido>

1. Introdução e objetivos

Este trabalho implementa uma **plataforma de chat em tempo real** com arquitetura **distribuída de microsserviços**, atendendo aos três pilares exigidos:

- **Alta disponibilidade** — o serviço de chat roda em múltiplas instâncias; a queda de uma instância não interrompe o serviço.
- **Comunicação em tempo real** — WebSockets ([Socket.IO](https://socket.io/)) entregam mensagens por *push* com baixa latência (medido: p95 $\approx$ 228 ms sob carga de 12 usuários simultâneos).
- **Escalabilidade horizontal** — novas instâncias do chat entram atrás de um balanceador NGINX, e o Redis garante a entrega de mensagens entre instâncias.

Toda a solução é escrita em **TypeScript** e orquestrada com **Docker Compose** — um único comando sobe o sistema completo.

2. Arquitetura e decisões de projeto



2.1 Divisão em microsserviços

A aplicação é dividida em dois serviços independentes, com bancos e ciclos de vida próprios:

Serviço	Responsabilidade	Banco
auth-service	Cadastro, login, emissão de JWT, listagem de usuários	PostgreSQL
chat-service	Envio, recebimento, armazenamento e transmissão em tempo real	MongoDB

Integração desacoplada por JWT: o **auth-service** **assina** o token no login e o **chat-service** o **valida** com o mesmo segredo compartilhado (**JWT_SECRET**). Não há chamada síncrona entre serviços a cada mensagem — a autenticação é *stateless*, decisão que favorece a escalabilidade.

2.2 Persistência poliglota

- **PostgreSQL (relacional)** para usuários: dados estruturados, com integridade (unicidade de **username**).

- **MongoDB (NoSQL)** para mensagens/conversas: otimizado para alta taxa de escrita e leitura de documentos.

2.3 Tempo real escalável (o problema do fan-out entre instâncias)

Com mais de uma instância de chat, dois usuários podem estar conectados a instâncias diferentes. Para que uma mensagem recebida pela instância 1 alcance um destinatário na instância 2, usamos o `@socket.io/redis-adapter`: cada instância publica/assina eventos num canal Redis, propagando o broadcast entre todas as réplicas. Esse é o mecanismo central da escalabilidade horizontal sem perder o tempo real.

2.4 Modelo de salas (rooms)

Cada usuário entra na sala `user:<userId>` ao conectar; cada conversa usa `conversation:<conversationId>`. No envio, a mensagem é persistida e emitida para a sala pessoal de **cada participante**, garantindo entrega **1:1** (`direct`) e **1:N** (`group`) independentemente da instância que atende cada cliente.

2.5 Gateway e balanceamento de carga

O **NGINX** é o ponto único de entrada (porta 8080). Como as rotas REST são *stateless*, elas usam **round-robin**; as conexões WebSocket (persistentes) usam `least_conn` (envia a nova conexão à instância com menos conexões ativas). Os clientes usam transporte *websocket-only*, dispensando *sticky sessions*; a entrega entre instâncias fica a cargo do Redis.

3. Tecnologias utilizadas

Camada	Tecnologia	Porta
Gateway / Balanceador	NGINX	8080
Autenticação	Node + TypeScript + Express	4000
Chat (×2 instâncias)	Express + Socket.IO	5000
Banco relacional	PostgreSQL	5432
Banco NoSQL	MongoDB	27017
Pub/Sub entre instâncias	Redis	6379
Front-end	React + Vite (TypeScript)	—

Bibliotecas principais: `bcryptjs` (hash de senha), `jsonwebtoken` (JWT), `pg`, `mongoose`, `socket.io` + `@socket.io/redis-adapter`, `axios`, `socket.io-client`.

4. Modelo de dados (armazenamento)

4.1 PostgreSQL — tabela `users`

Coluna	Tipo	Observação
<code>id</code>	<code>uuid</code> (PK)	gerado pelo banco
<code>username</code>	<code>text</code> UNIQUE	não pode repetir
<code>password_hash</code>	<code>text</code>	hash bcrypt (a senha nunca é armazenada em texto)
<code>created_at</code>	<code>timestampz</code>	data de criação

4.2 MongoDB — collections `conversations` e `messages`

```
conversations: { _id, type:"direct"|"group", name?, participantIds[],
                 lastMessage:{content,senderId,createdAt}, createdAt, updatedAt }
messages:      { _id, conversationId, senderId, senderUsername, content, createdAt }
```

Conversas `direct` são **reutilizadas** (não duplicam a DM entre os mesmos dois usuários). `participantIds` e `conversationId` são indexados.

4.3 Fluxo de envio de mensagem (`message:send`)

1. **Autoriza** — confere se o remetente é participante (senão, erro 403).
2. **Persiste** — grava o documento em `messages`.
3. **Atualiza a conversa** — grava `lastMessage` / `updatedAt`.
4. **Broadcast** — emite `message:new` para a sala de cada participante; o Redis propaga para participantes em outras instâncias.

*A mensagem é gravada **antes** de ser entregue — por isso o histórico funciona mesmo para quem estava offline no momento do envio.*

5. Funcionalidades implementadas

- Cadastro e login com hash bcrypt e autenticação JWT.
- Conversas **1:1** (privadas) e em **grupo 1:N**.
- Entrega de mensagens em **tempo real** (WebSockets).
- **Histórico** persistido (MongoDB) com paginação.
- Indicador de digitação e presença online.
- Interface **responsiva** (desktop e mobile).
- **Escalabilidade horizontal** (2+ instâncias + Redis) e **balanceamento** (NGINX).

6. Plano de testes e avaliação

Os testes cobrem os três níveis exigidos pelo enunciado.

6.1 Testes unitários (lógica de negócio central)

- **auth-service:** validação de credenciais, hash de senha, emissão/verificação de JWT, rejeição de usuário duplicado — sem necessidade de Postgres (repositório mockado).
- **chat-service:** persistência correta de mensagens, reuso de DM, criação de grupo e autorização de participante — usando `mongodb-memory-server`.

6.2 Testes de integração

- **chat-service (`socket.integration`):** dois clientes autenticados; A envia, B recebe em tempo real e a mensagem é persistida.
- **ponta-a-ponta (`e2e`):** exercita os **dois serviços juntos** — "*autenticar e, em seguida, enviar uma mensagem*" — cobrindo 1:1, 1:N, persistência e autorização (403).

6.3 Teste de concorrência/carga

- **load :** 12 usuários simultâneos (o PDF exige ≥ 10) autenticam, abrem conexões WebSocket em paralelo e trocam mensagens em rajada num grupo 1:N. Mede taxa de entrega, latência (p50/p95/p99) e vazão.
- **1b :** evidencia a distribuição de requisições entre as instâncias.

7. Evidências de funcionamento

*Execuções reais coletadas em Docker Desktop (Windows 11), com a stack completa no ar e **duas instâncias** de chat.*

7.1 Infraestrutura — 8 containers no ar

SERVICE	STATUS	PORTS
auth-service	Up	0.0.0.0:4000->4000/tcp
chat-service-1	Up	5000/tcp
chat-service-2	Up	5000/tcp
frontend	Up	80/tcp
gateway	Up	0.0.0.0:8080->80/tcp
mongo	Up (healthy)	0.0.0.0:27017->27017/tcp
postgres	Up (healthy)	0.0.0.0:5432->5432/tcp
redis	Up (healthy)	0.0.0.0:6379->6379/tcp

Saúde das duas instâncias (cada uma responde com seu identificador):

```
{"status":"ok","instance":"chat-1"}  
{"status":"ok","instance":"chat-2"}
```

7.2 Testes unitários — auth-service (20/20)

```
PASS tests/routes.test.ts  
PASS tests/authService.unit.test.ts  
Test Suites: 2 passed, 2 total  
Tests:      20 passed, 20 total
```

7.3 Testes unitários/integração — chat-service (24/24)

```
PASS tests/socket.integration.test.ts  
PASS tests/routes.test.ts  
PASS tests/messageService.unit.test.ts  
PASS tests/conversationService.unit.test.ts  
Test Suites: 4 passed, 4 total  
Tests:      24 passed, 24 total
```

7.4 Integração ponta-a-ponta (auth ⇌ chat) — 16/16

```
[1] Autenticação no auth-service  
  ✓ Alice obteve token JWT  
  ✓ Bob obteve token JWT  
  ✓ GET /api/users retornou outros usuários  
  ✓ Lista não inclui o próprio requester  
[2] Conexão WebSocket no chat-service usando o JWT do auth-service  
  ✓ Socket de Alice conectado (JWT aceito pelo chat)  
  ✓ Socket de Bob conectado  
  ✓ Conexão com JWT inválido é rejeitada  
[3] Mensagem 1:1 em tempo real + persistência  
  ✓ Conversa direta criada  
  ✓ Conversa direta é reutilizada (sem duplicar)  
  ✓ Bob recebeu a mensagem em tempo real  
  ✓ Mensagem traz o username do remetente  
  ✓ Mensagem persistida no histórico (MongoDB)  
[4] Mensagem 1:N (grupo) em tempo real  
  ✓ Grupo criado com 3 participantes  
  ✓ Bob recebeu a mensagem do grupo  
  ✓ Carol recebeu a mensagem do grupo (1:N)  
[5] Autorização  
  ✓ Não-participante recebe 403 ao ler conversa alheia  
===  TODOS os testes de integração passaram ===
```

7.5 Teste de concorrência/carga — 12 usuários simultâneos

```
=== Teste de Carga: 12 usuários simultâneos, 5 msgs/usuário ===
  ✓ 12 usuários autenticados em 2490 ms
  ✓ 12 sockets conectados em 90 ms
→ Grupo 1:N criado com 12 participantes
→ Enviando 60 mensagens em rajada...
===== RESULTADO =====
Usuários simultâneos.....: 12
Mensagens enviadas.....: 60
Entregas esperadas (fan-out): 720
Entregas recebidas.....: 720
Taxa de entrega.....: 100.00%
Vazão (entregas/s).....: 1516
Latência p50/p95/p99 (ms).: 106 / 228 / 244
=====
✅ SUCESSO: todas as mensagens entregues a todos os participantes em tempo real.
```

7.6 Balanceamento de carga — distribuição entre instâncias

```
=== Verificação de Balanceamento de Carga (40 requisições via gateway) ===
Distribuição das respostas por instância:
chat-1          20 ██████████
chat-2          20 ██████████
✅ Tráfego distribuído entre 2 instâncias – balanceamento de carga confirmado.
```

7.7 Persistência nos bancos

PostgreSQL (`users`) e MongoDB (`conversations` / `messages`) após as execuções:

```
-- PostgreSQL: usuários cadastrados
  username      |      created_at
-----+-----
e2e_966dra_alice | 2026-06-26 12:27:26.85+00
e2e_966dra_bob   | 2026-06-26 12:27:27.03+00
load_87slhv_u9   | 2026-06-26 12:27:31.76+00
...

-- MongoDB: documentos persistidos
conversations: 7
messages: 130
```

7.8 Síntese dos resultados

Teste	Resultado
Unitários auth-service	20/20 ✓ (≈94% cobertura)
Unitários/integração chat-service	24/24 ✓
Integração ponta-a-ponta (e2e)	16/16 ✓
Carga (12 usuários)	720/720 entregas (100%) , p95 = 228 ms, ≈1.516 entregas/s
Balanceamento	20 chat-1 / 20 chat-2 ✓

8. Como executar

Pré-requisito: **Docker Desktop** em execução.

```
# Subir o sistema completo (8 containers)
cd chat-distribuido
docker compose up --build -d
docker compose ps          # conferir status

# Acessar a aplicação
# http://localhost:8080
```

Para testar **duas contas simultâneas**, use uma janela normal + uma janela anônima (**Ctrl+Shift+N**). Em uma delas, clique em "+ Nova conversa": selecione um usuário (1:1) ou vários + nome do grupo (1:N).

Testes:

```
cd services/auth-service && npm test          # 20 testes
cd services/chat-service && npm test          # 24 testes
cd load-tests && npm run e2e && npm run load && npm run lb
```

Demonstrar alta disponibilidade:

```
docker compose stop chat-service-1          # o chat segue ativo na instância 2
```

9. Conclusão

A solução atende a todos os requisitos mínimos: arquitetura distribuída de microsserviços, comunicação assíncrona em tempo real por WebSockets, persistência poliglota (PostgreSQL + MongoDB), mensagens 1:1 e 1:N com histórico, interface responsiva, além dos três tipos de teste exigidos — todos com evidências reais de execução. As decisões de projeto (JWT *stateless*, Redis adapter para o fan-out entre instâncias e NGINX como balanceador) foram tomadas visando especificamente **alta disponibilidade**,

baixa latência e escalabilidade horizontal, comprovadas pelo teste de carga (100% de entrega a 12 usuários simultâneos com distribuição entre as instâncias).